



HOPPER INFORMATION SERIES

TEACHABLE MACHINE

Teachable Machine models

Build mission-specific whole-frame labels, export the browser model, and validate it honestly.

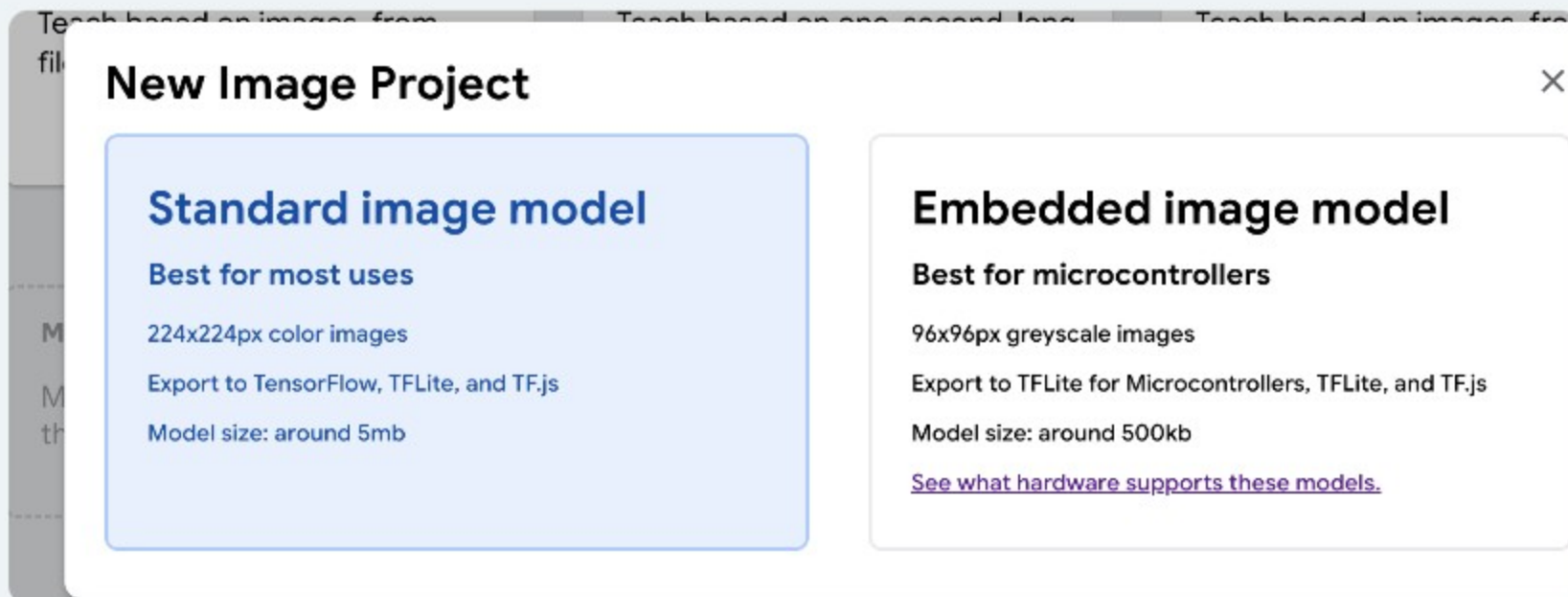
224x224 RGB

TRANSFER
LEARNING

CUSTOM LABELS



Choose a standard image model



For Hopper Studio

- Select Image Project.
- Choose Standard image model.
- Export TensorFlow.js files after training.
- Keep model.json, weights.bin, and metadata.json together.

Use Standard image model: 224x224 color input and TensorFlow.js export. The embedded option targets microcontrollers.

Custom labels are the advantage — “other” is the guardrail

RED FLAG

Red fabric, multiple
angles

BLUE FLAG

Blue fabric, multiple
distances

SAFE PAD

Target pad under flight
lighting

OTHER / BACKGROUND

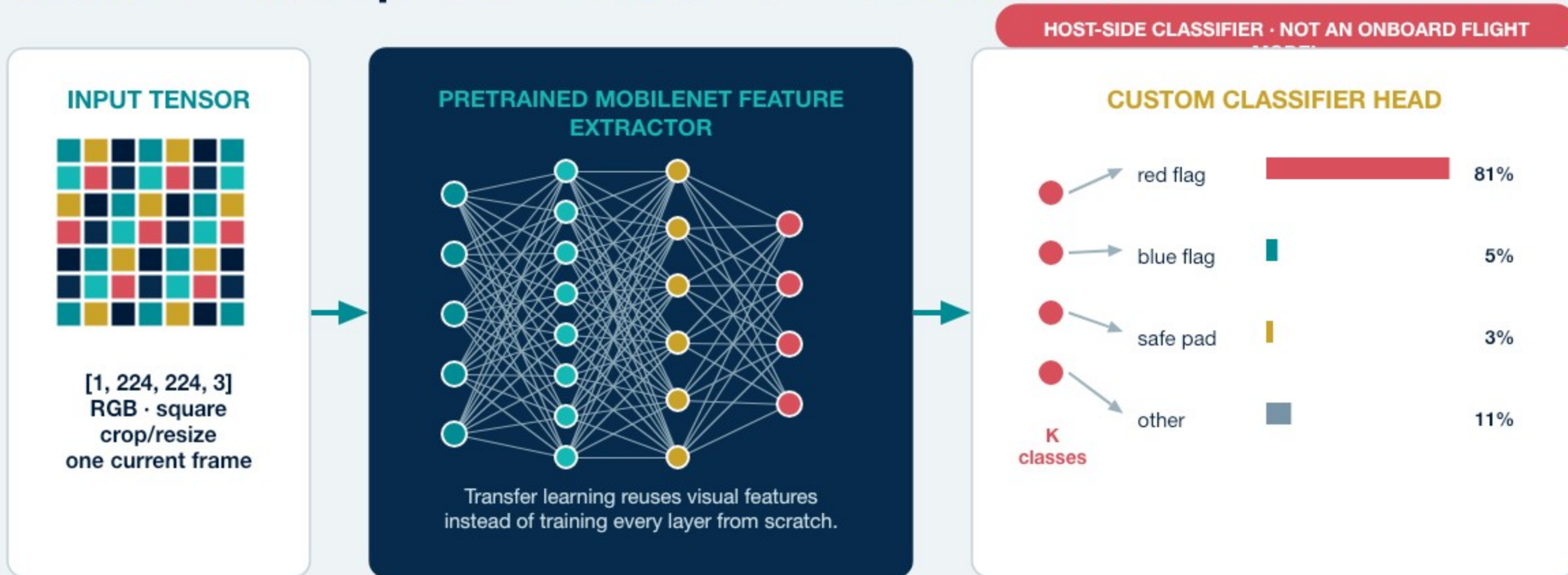
Floor, walls, people,
blur, empty room

Why the “other” class matters

- A classifier must distribute probability across the labels it knows.
- Without negative/background examples, an irrelevant frame can be forced into a mission label.
- Make “other” diverse: empty views, different floors, students, walls, shadows, motion blur, and anything you do not want to trigger.

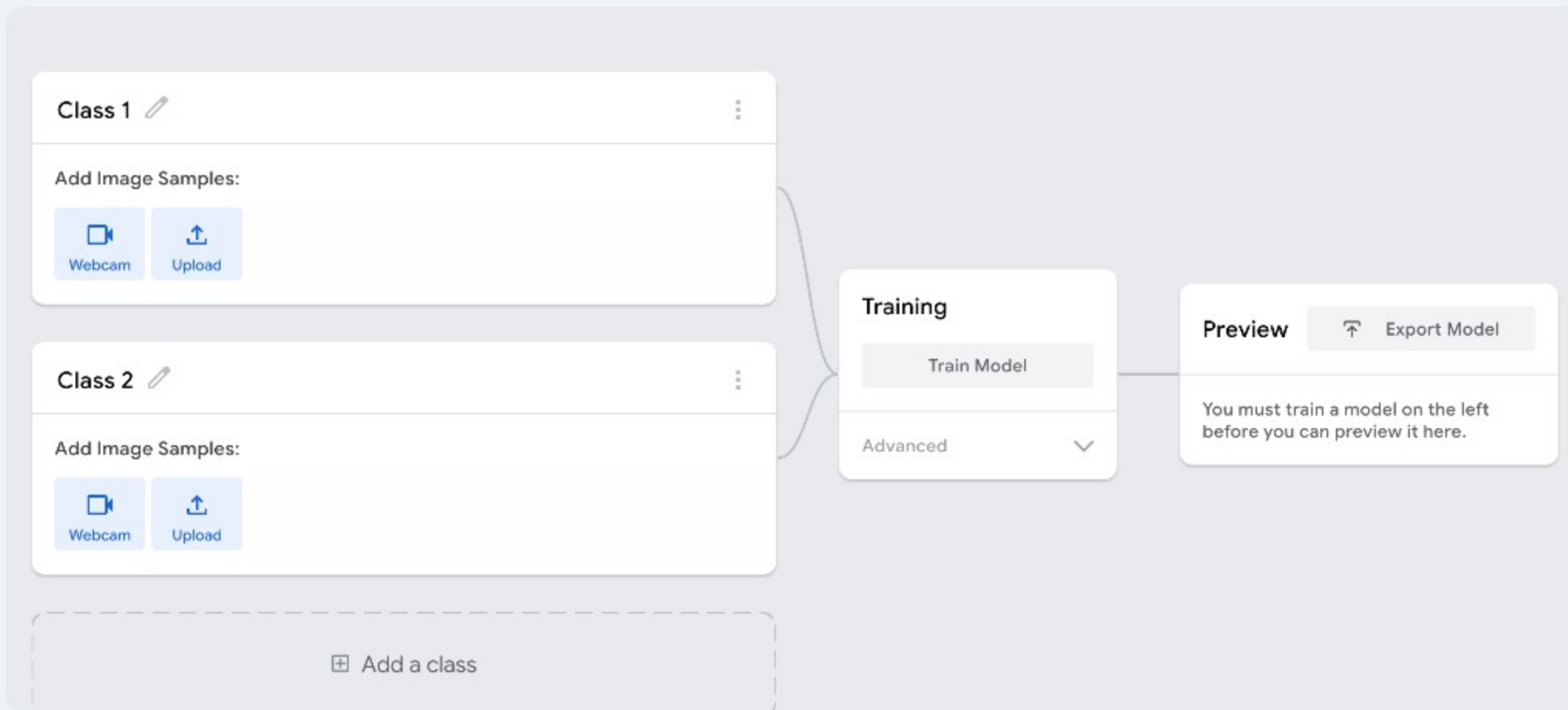
Choose labels that answer one mission question — and collect the world outside those labels.

Custom model: pixels $\rightarrow$ features $\rightarrow$ class scores



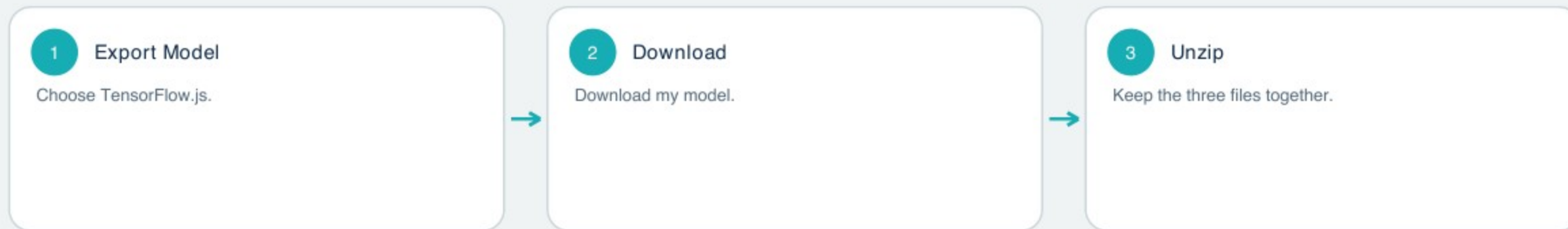
Exact layer shapes and coefficient count depend on the exported model and number of classes—inspect model.json.

Use the training screen as an experiment dashboard

**1 · COLLECT BALANCED CLASSES****2 · TRAIN****3 · PREVIEW UNSEEN VIEWS**



Export the standard TensorFlow.js files



`hopper-model/`

`model.json` network structure

`weights.bin` learned parameters

`metadata.json` labels + project metadata

Not supported: .tflite, embedded export or only one of the three files.



Load the model into Hopper Studio



IMPORTANT: THE MODEL IS NOT FLASHED INTO THE AIRCRAFT

Hopper Studio loads the files into browser/app memory and runs inference on this computer.

Reload after refresh. The model files are not persisted or uploaded to a cloud service.

MODEL.JSON

WEIGHTS.BIN

METADATA.JSON

Ask for one label, one threshold, and one safe outcome

CUSTOM LABEL DECISION · PROGRAM.JS

```
1 await drone.takeOff();
2 try {
3   const redFlag = await vision.seesCustomLabel(
4     "red flag",
5     0.80,
6   );
7
8   if (redFlag) {
9     await drone.fly("forward", 1, 12);
10  }
11 } finally {
12   await drone.land();
13 }
```

CUSTOM CLASSIFIER

```
await vision.seesCustomLabel(label, minimumConfidence)
```

label — string; must match an exported class name

minimumConfidence — 0...1 score; default 0.75

returns — Promise<boolean> after a fresh whole-frame prediction

throws — if the 3 model files are not loaded or pixels are blocked

scope — one class decision; no box or x/y coordinate

FLIGHT COMMAND USED ABOVE

```
await drone.fly(direction, seconds, power)
```

direction — required direction string; example forward

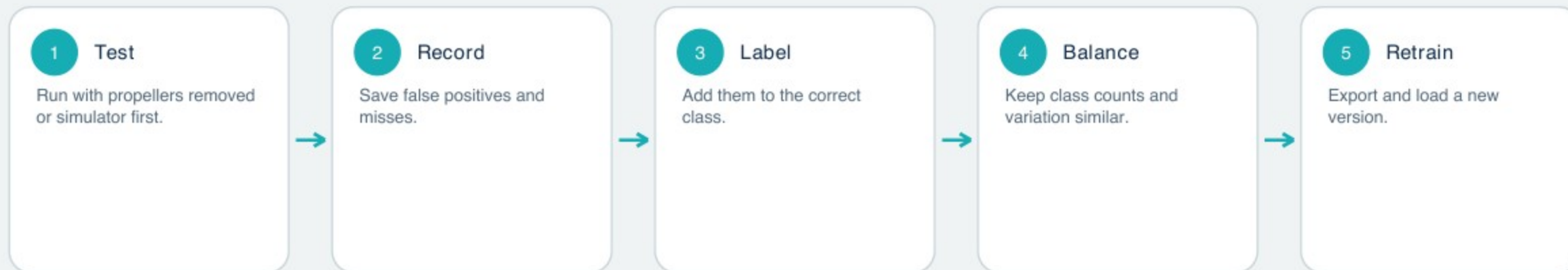
seconds — duration ≥ 0 ; example 1 second

power — signed percentage clamped to -100...100;
example 12

returns — Promise<void>; await before landing or another command



Treat every failure as new training data



FLIGHT ACCEPTANCE TEST

- Target class succeeds across distance/rotation.
- Negative class rejects empty floor.
- No single lighting condition dominates.
- Threshold chosen from measured errors.

SOURCES AND FURTHER READING

Google Teachable Machine

<https://teachablemachine.withgoogle.com/train>

TensorFlow.js transfer learning overview

<https://codelabs.developers.google.com/tensorflowjs-transfer-learning-teachable-machine>

Hopper Studio custom-model implementation

[lib/vision.ts](#) and [README.md](#)

A custom model is only ready after an honest field test

PASS CRITERIA

- Correct label across expected distance, angle, and light.
- Low false-trigger rate on the “other” class.
- Stable predictions during short motion blur and camera changes.
- Works on held-out examples that were never used in training.

IF IT FAILS

- Record the failure view.
- Label it correctly—often as other/background.
- Rebalance classes so one does not dominate.
- Retrain, export all three files, and repeat the same test.

Custom labels are powerful because they match your mission. The cost is owning the data, the negative class, and the validation.